

Mobile Development

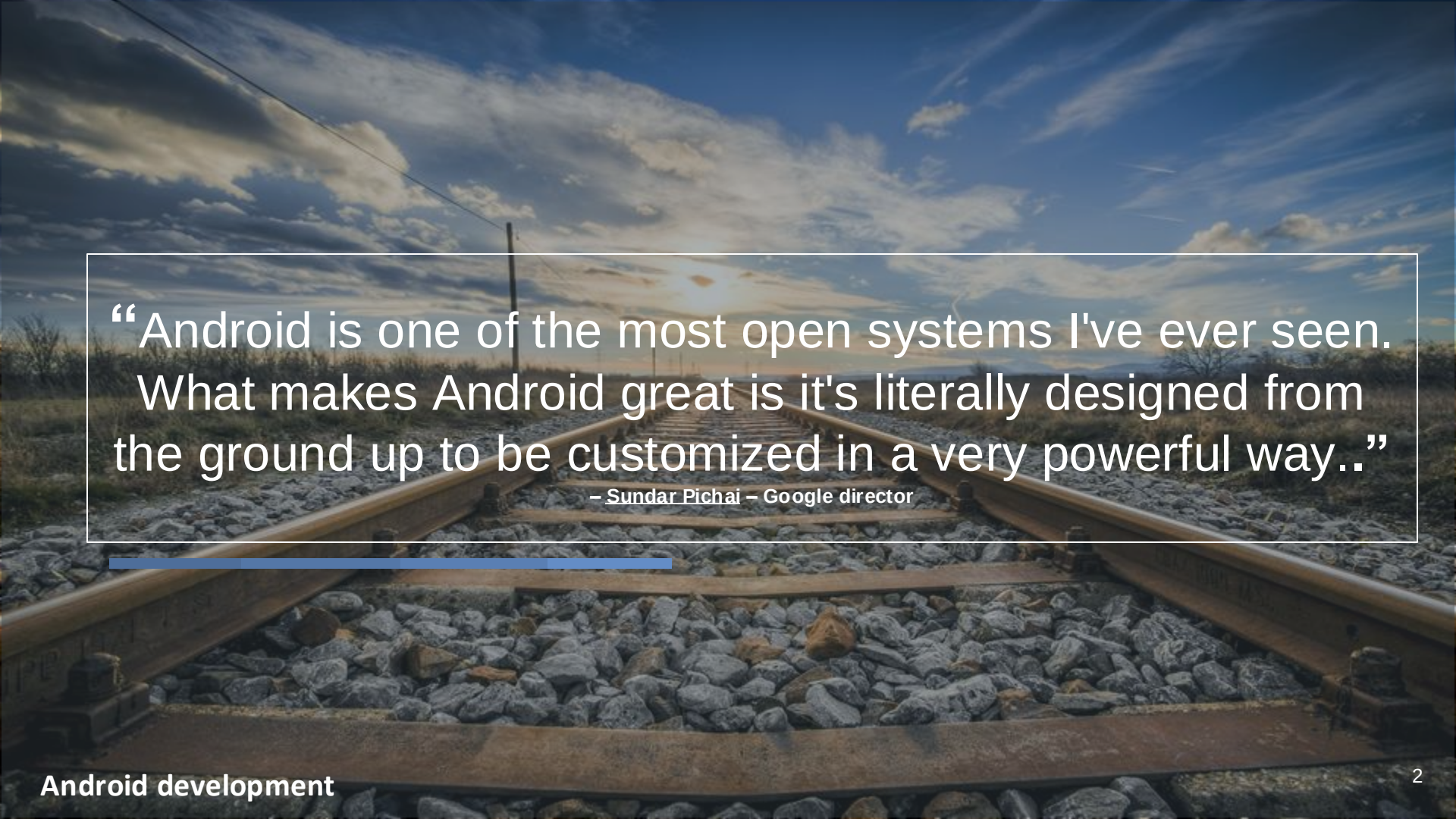
(Lab 3)

Instructor: Tran Vinh Khiem

March 1st, 2022



Smart Software System Team



“Android is one of the most open systems I've ever seen. What makes Android great is it's literally designed from the ground up to be customized in a very powerful way..”

– Sundar Pichai – Google director



Learning Objectives

- Provide you with realistic, hands-on experience developing Android applications.
- Create a portfolio of apps that you can show your friends, discuss in interviews, and borrow for other applications.

Adapter



- **adapter**: Helps turn list data into list view items.
 - common adapters: ArrayAdapter, CursorAdapter

- Syntax for creating an adapter:

```
ArrayAdapter<String> name =  
    new ArrayAdapter<String>(activity, layout, array);
```

- the ***activity*** is usually this
 - the default ***layout*** for lists is `android.R.layout.simple_list_item_1`
 - get the ***array*** by reading your file or data source of choice
(it can be an array like `String[]`, or a list like `ArrayList<String>`)
- Once you have an adapter, you can attach it to your list by calling the `setAdapter` method of the `ListView` object in the Java code.

List Adapter

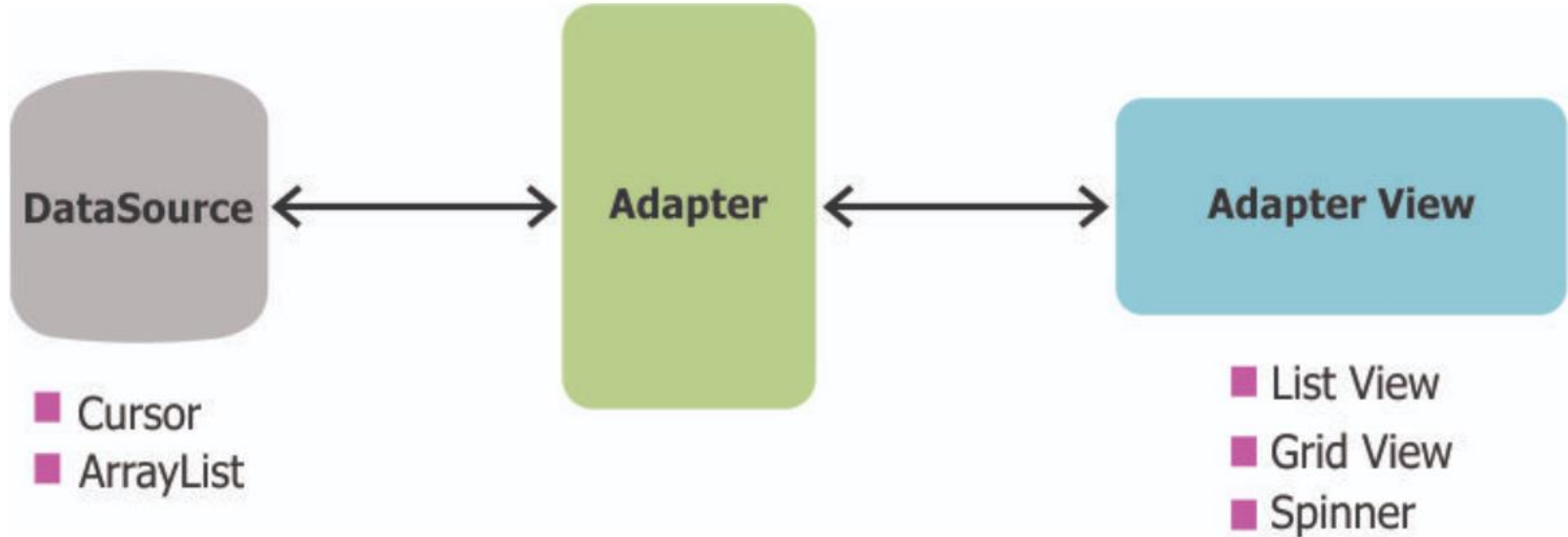


```
ArrayList<String> myArray = ...; // load data from file
```

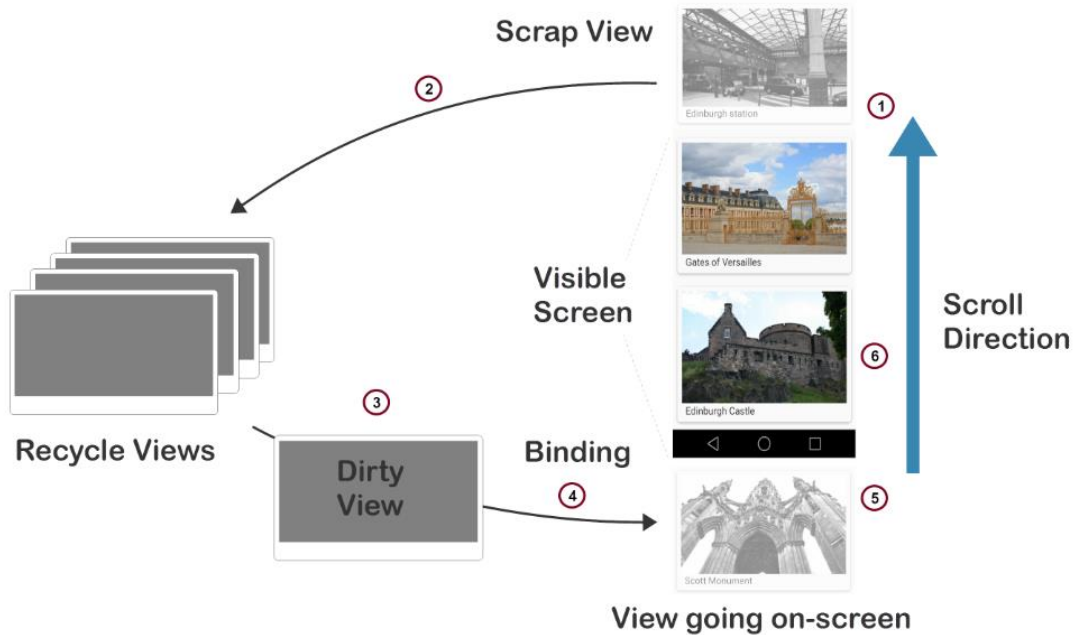
```
ArrayAdapter<String> adapter =  
    new ArrayAdapter<String>(  
        this,  
        android.R.layout.simple_list_item_1,  
        myArray);
```

```
ListView list = (ListView) findViewById(R.id.mylist);  
list.setAdapter(myAdapter);
```

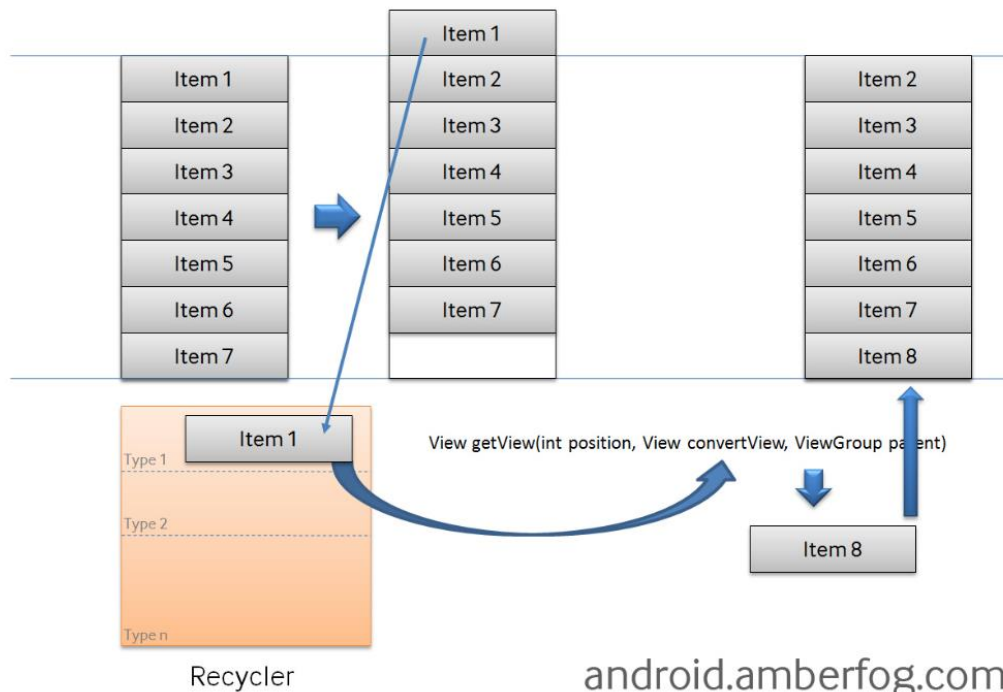
List Adapter



Row View Recycling



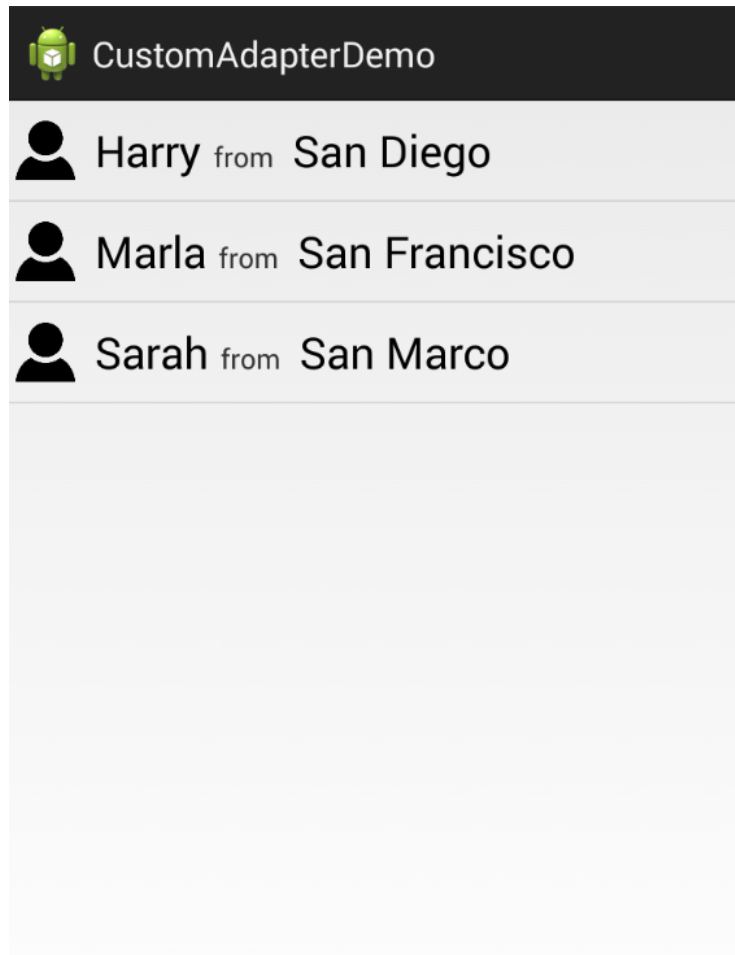
Row View Recycling



android.amberfog.com

Exercise

- Create this application with Adapter





Exercise

```
ArrayAdapter<String> itemsAdapter =  
    new ArrayAdapter<String>(this, android.R.layout.simple_list_item_1, items);
```

```
ListView listView = (ListView) findViewById(R.id.lvItems);  
listView.setAdapter(itemsAdapter);
```



Exercise - Defining a model

```
public class User {  
    public String name;  
    public String hometown;  
  
    public User(String name, String hometown) {  
        this.name = name;  
        this.hometown = hometown;  
    }  
}
```



Exercise - Custom Array Adapter – Creating the View Template

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >
    <TextView
        android:id="@+id/tvName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Name" />
    <TextView
        android:id="@+id/tvHome"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="HomeTown" />
</LinearLayout>
```

Exercise - Custom Array Adapter – Defining adapter



```
public class UsersAdapter extends ArrayAdapter<User> {
    public UsersAdapter(Context context, ArrayList<User> users) {
        super(context, 0, users);
    }

    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        // Get the data item for this position
        User user = getItem(position);
        // Check if an existing view is being reused, otherwise inflate the view
        if (convertView == null) {
            convertView = LayoutInflater.from(getContext()).inflate(R.layout.item_user, parent, false);
        }
        // Lookup view for data population
        TextView tvName = (TextView) convertView.findViewById(R.id.tvName);
        TextView tvHome = (TextView) convertView.findViewById(R.id.tvHome);
        // Populate the data into the template view using the data object
        tvName.setText(user.name);
        tvHome.setText(user.hometown);
        // Return the completed view to render on screen
        return convertView;
    }
}
```

Exercise - Custom Array Adapter – Attaching the Adapter to ListView

```
// Construct the data source  
ArrayList<User> arrayOfUsers = new ArrayList<User>();  
// Create the adapter to convert the array to views  
UsersAdapter adapter = new UsersAdapter(this, arrayOfUsers);  
// Attach the adapter to a ListView  
ListView listView = (ListView) findViewById(R.id.lvItems);  
listView.setAdapter(adapter);
```

Exercise - Custom Array Adapter – Populating Data into ListView



```
// Add item to adapter  
User newUser = new User("Nathan", "San Diego");  
adapter.add(newUser);  
// Or even append an entire new collection  
// Fetching some data, data has now returned  
// If data was JSON, convert to ArrayList of User objects.  
JSONArray jsonArray = ...;  
ArrayList<User> newUsers = User.fromJson(jsonArray)  
adapter.addAll(newUsers);
```

```
adapter.clear();
```

Exercise - Custom Array Adapter – Constructing Models from External Source



```
public class User {  
    // Constructor to convert JSON object into a Java class instance  
    public User(JSONObject object){  
        try {  
            this.name = object.getString("name");  
            this.hometown = object.getString("hometown");  
        } catch (JSONException e) {  
            e.printStackTrace();  
        }  
    }  
  
    // Factory method to convert an array of JSON objects into a list of objects  
    // User.fromJson(jsonArray);  
    public static ArrayList<User> fromJson(JSONArray jsonObjects) {  
        ArrayList<User> users = new ArrayList<User>();  
        for (int i = 0; i < jsonObjects.length(); i++) {  
            try {  
                users.add(new User(jsonObjects.getJSONObject(i)));  
            } catch (JSONException e) {  
                e.printStackTrace();  
            }  
        }  
        return users;  
    }  
}
```




Exercise - Custom Array Adapter – Attaching Event Handlers Within Adapter

```
public class UsersAdapter extends ArrayAdapter<User> {  
    // ...  
    @Override  
    public View getView(int position, View convertView, ViewGroup parent) {  
        // ...  
        // Lookup view for data population  
        Button btButton = (Button) convertView.findViewById(R.id.btButton);  
        // Cache row position inside the button using `setTag`  
        btButton.setTag(position);  
        // Attach the click event handler  
        btButton.setOnClickListener(new View.OnClickListener() {  
            @Override  
            public void onClick(View view) {  
                int position = (Integer) view.getTag();  
                // Access the row position here to get the correct data item  
                User user = getItem(position);  
                // Do what you want here...  
            }  
        });  
        // ... other view population as needed...  
        // Return the completed view  
        return convertView;  
    }  
}
```



Exercise - Custom Array Adapter – Attaching Event Handlers Within Adapter

```
// Inside adapter `getView` method
User user = getItem(position);
// Cache user object inside the button using `setTag`
btButton.setTag(user);
// Attach the click event handler
btButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        // Access user from within the tag
        User user = (User) view.getTag();
        // Do what you want here...
    }
});
```



Exercise - Custom Array Adapter – Improving Performance with the ViewHolder Pattern

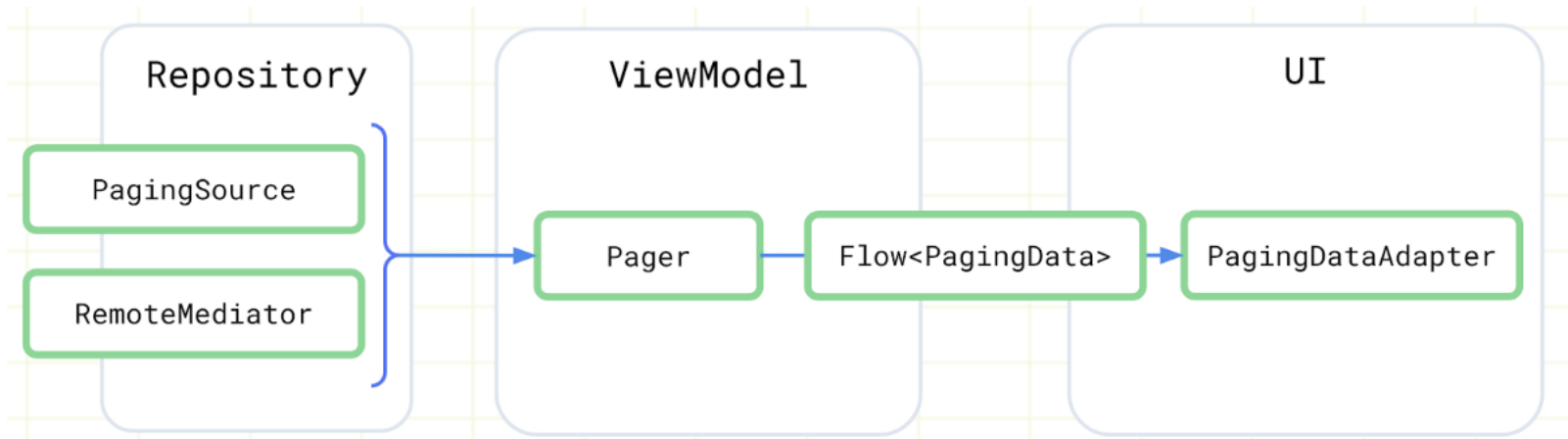
```
public class UsersAdapter extends ArrayAdapter<User> {  
    // View lookup cache  
    private static class ViewHolder {  
        TextView name;  
        TextView home;  
    }  
  
    public UsersAdapter(Context context, ArrayList<User> users) {  
        super(context, R.layout.item_user, users);  
    }  
}
```



Exercise - Custom Array Adapter – Improving Performance with the ViewHolder Pattern

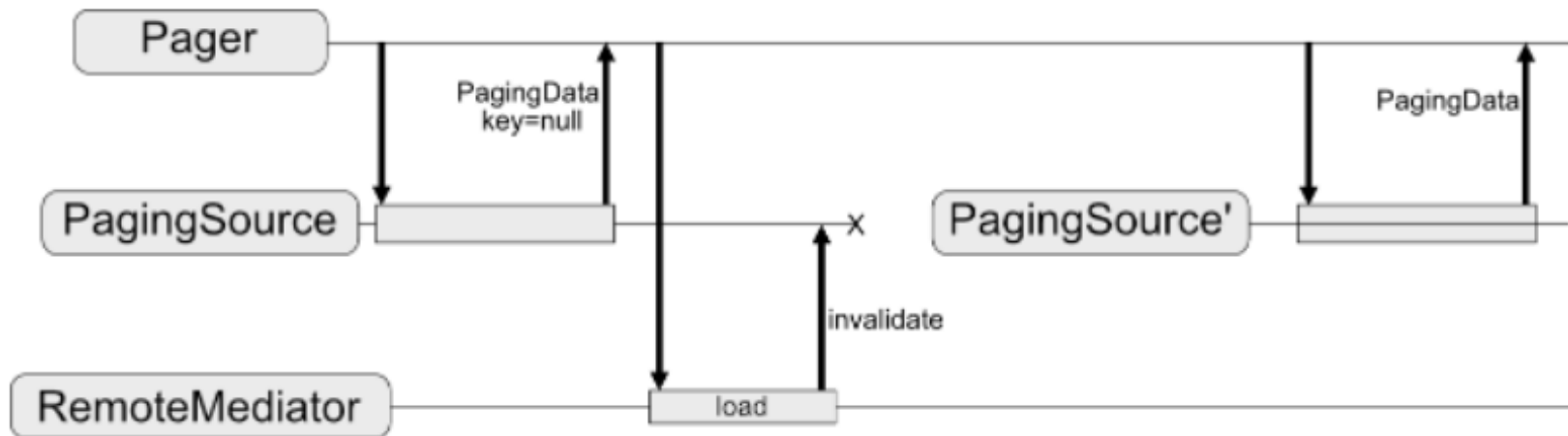
```
@Override
public View getView(int position, View convertView, ViewGroup parent) {
    // Get the data item for this position
    User user = getItem(position);
    // Check if an existing view is being reused, otherwise inflate the view
    ViewHolder viewHolder; // view lookup cache stored in tag
    if (convertView == null) {
        // If there's no view to re-use, inflate a brand new view for row
        viewHolder = new ViewHolder();
        LayoutInflater inflater = LayoutInflater.from(getContext());
        convertView = inflater.inflate(R.layout.item_user, parent, false);
        viewHolder.name = (TextView) convertView.findViewById(R.id.tvName);
        viewHolder.home = (TextView) convertView.findViewById(R.id.tvHome);
        // Cache the viewHolder object inside the fresh view
        convertView.setTag(viewHolder);
    } else {
        // View is being recycled, retrieve the viewHolder object from tag
        viewHolder = (ViewHolder) convertView.getTag();
    }
    // Populate the data from the data object via the viewHolder object
    // into the template view.
    viewHolder.name.setText(user.name);
    viewHolder.home.setText(user.hometown);
    // Return the completed view to render on screen
    return convertView;
}
```

Paging

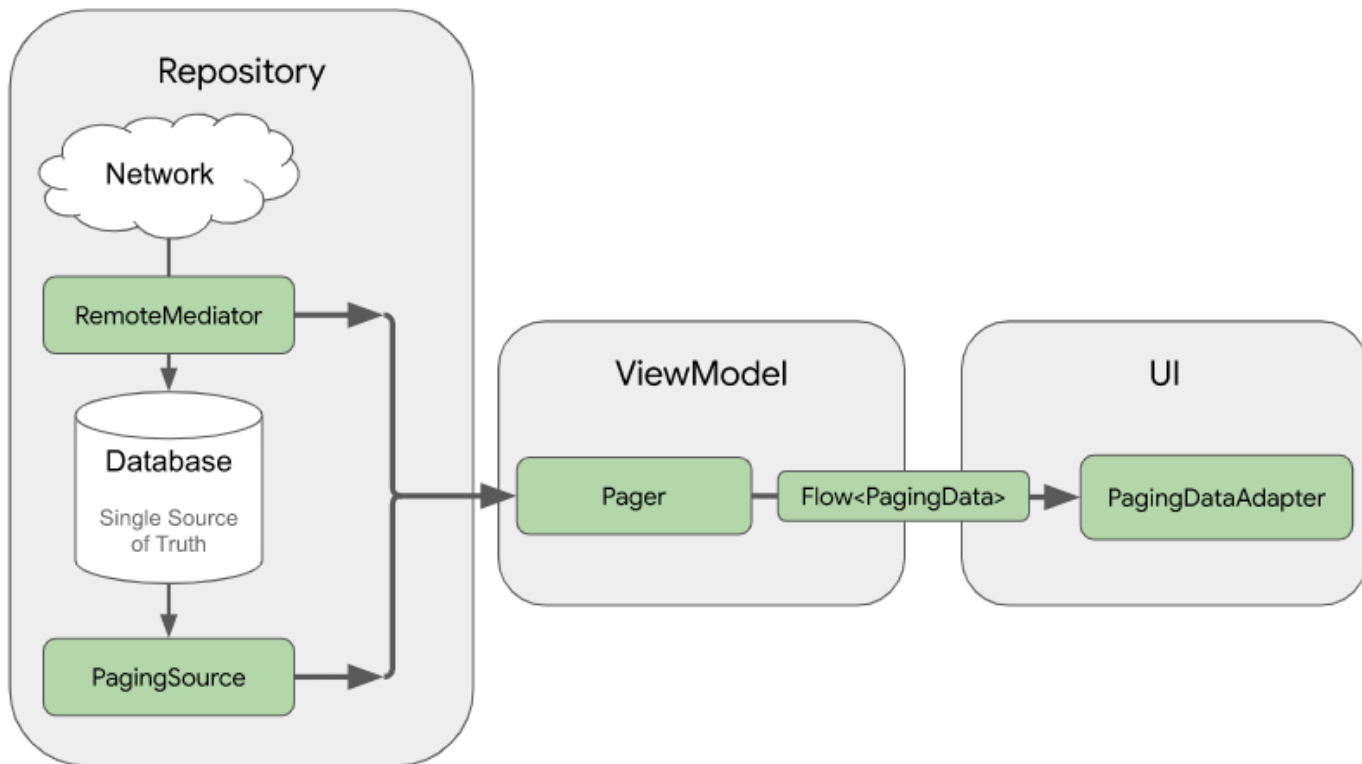


Paging components and their integration in the app architecture"

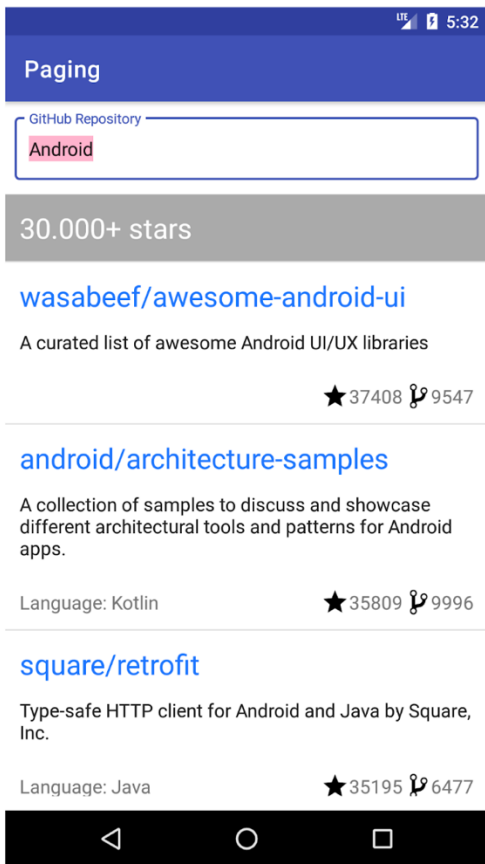
Paging



Paging



Paging – Example

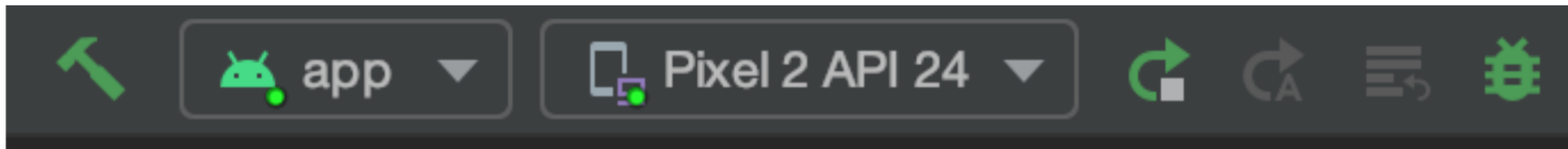


Paging – Set up



```
git clone https://github.com/googlecode-labs/android-paging
```

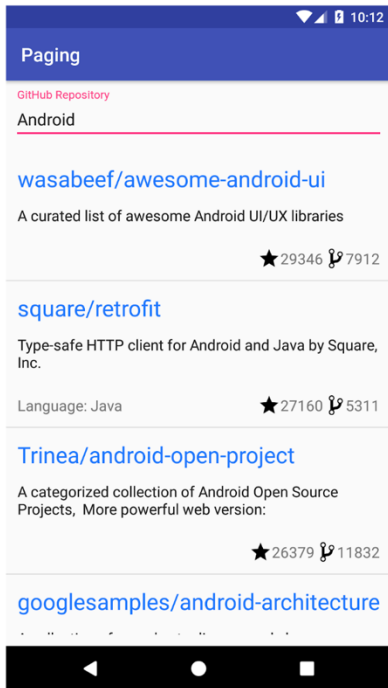
1. Unzip the code, and then open the project in Android Studio.
2. Run the `app` run configuration on a device or emulator.



Paging – Set up Exercise



The app runs and displays a list of GitHub repositories similar to this one:



Retrofit



- [Retrofit](#) is a type-safe REST client for Android, Java and Kotlin developed by Square. The library provides a powerful framework for authenticating and interacting with APIs and sending network requests with [OkHttp](#).



Retrofit – Set up - Exercise

Make sure to require Internet permissions in your `AndroidManifest.xml` file:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET" />
</manifest>
```

Add the following to your `app/build.gradle` file:

```
dependencies {
    implementation 'com.google.code.gson:gson:2.8.7'
    implementation 'com.squareup.retrofit2:retrofit:2.9.0'
    implementation 'com.squareup.retrofit2:converter-gson:2.9.0'
}
```



Retrofit – CodeLab - Exercise

- Do this codelab with Java and take screenshot
- <https://developer.android.com/codelabs/basic-android-kotlin-training-getting-data-internet?hl=en#0>
- Replace with your API and do it again



Paging – CodeLab - Exercise

- Do this codelab with Java and take screenshot
- <https://developer.android.com/codelabs/android-paging#2>

Exercise 1

- Access to newsapi.org
- Create news paging application with this API.
- Example of this app is on the picture.

News

UK weather forecast: Bank holiday weekend washout but Indian summer could then last
High temperatures could continue into November, according to long-range Met Office forecasts

Mirror

🕒 21:47 on Aug 25, 2018

Crowds brave the rain at Leeds Festival for spectacular Fall Out Boy show

American rockers Fall Out Boy wowed fans with flame-throwing guitars and dazzling pyrotechnics during the second headline performance at Leeds F...

Independent.ie

🕒 21:41 on Aug 25, 2018

Maharashtra: Paw amputated in 2012, tiger to get artificial limb soon

On Thursday, a team of government veterinary doctors — Shirish Upadhye, Vinod Dhoot and Gautam Bhojne — examined Sahebrao at Gorewada centre, ...

Indianexpress.com

🕒 21:29 on Aug 25, 2018

World Triathlon Series: Vicky Holland wins gold in Montreal to close in on title

Britain's Vicky Holland claims gold in Montreal to close the gap on Katie Zaferes at the top of the World Triathlon Series standings.

BBC News

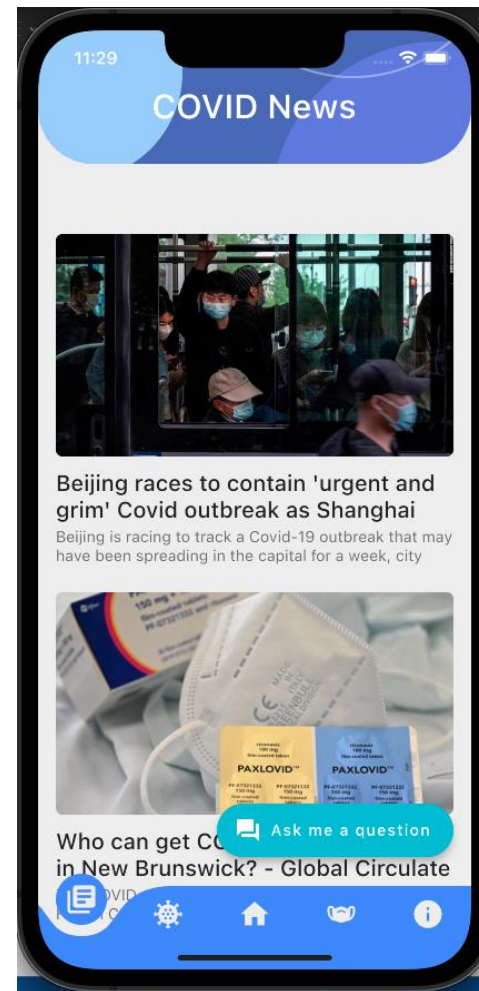
🕒 21:07 on Aug 25, 2018

CHAMPIONSHIP ROUND-UP: Leeds no top

Powered by News API

Exercise 1

- Access to newsapi.org
- Create news paging application with this API.
- Example of this app is on the picture.



Exercise 1 – Advance with news sentiment analysis



Data class:

```
data class NewsResponse(  
    val articles: List<Article>,  
    val totalResults: Int  
)  
  
data class Article(  
    val title: String,  
    val content: String?,  
    val urlToImage: String?,  
    val publishedAt: String  
)
```

Exercise 1 – Advance with sentiment analysis



Build.gradle

```
// Paging
implementation "androidx.paging:paging-runtime-ktx:3.2.1"

// Retrofit
implementation "com.squareup.retrofit2:retrofit:2.9.0"
implementation "com.squareup.retrofit2:converter-gson:2.9.0"

// TensorFlow Lite
implementation "org.tensorflow:tensorflow-lite-task-text:0.4.4"

// Coil for images
implementation "io.coil-kt:coil:2.5.0"
```

Exercise 1 – Advance with sentiment analysis



Retrofit

```
interface NewsApiService {  
    @GET("v2/everything")  
    suspend fun getArticles(  
        @Query("q") query: String,  
        @Query("pageSize") pageSize: Int,  
        @Query("page") page: Int,  
        @Query("apiKey") apiKey: String  
    ): NewsResponse  
}
```

Download pre-trained sentiment analysis model from TF Hub:

<https://www.tensorflow.org/hub?hl=vi>

Place sentiment_model.tflite in app/src/main/assets

Exercise 1 – Advance with sentiment analysis



Sentiment analysis

```
class SentimentAnalyzer(context: Context) {
    private val classifier: TextClassifier

    init {
        val model = FileUtil.loadMappedFile(context, "sentiment_model.tflite")
        val options = TextClassifier.TextClassifierOptions.builder().build()
        classifier = TextClassifier.createFromBufferAndOptions(model, options)
    }

    fun analyze(text: String): SentimentResult {
        val results = classifier.classify(text)
        return SentimentResult(
            positive = results.classificationResults[0].categories[0].score,
            negative = results.classificationResults[0].categories[1].score
        )
    }
}

data class SentimentResult(val positive: Float, val negative: Float)
```

Exercise 1 – Advance with sentiment analysis



Paging

```
class NewsPagingSource(
    private val service: NewsApiService,
    private val apiKey: String,
    private val analyzer: SentimentAnalyzer
) : PagingSource<Int, ArticleWithSentiment>() {

    override suspend fun load(params: LoadParams<Int>): LoadResult<Int, ArticleWithSentiment> {
        return try {
            val page = params.key ?: 1
            val response = service.getArticles(
                query = "android",
                pageSize = params.loadSize,
                page = page,
                apiKey = apiKey
            )

            val articlesWithSentiment = response.articles.map { article ->
                val sentiment = analyzer.analyze(article.content ?: article.title)
                ArticleWithSentiment(article, sentiment)
            }

            LoadResult.Page(
                data = articlesWithSentiment,
                prevKey = if (page == 1) null else page - 1,
                nextKey = if (page * params.loadSize < response.totalResults) page + 1 else null
            )
        } catch (e: Exception) {
            LoadResult.Error(e)
        }
    }
}
```

Exercise 1 – Advance with sentiment analysis



View model

```
class NewsViewModel(  
    private val service: NewsApiService,  
    private val analyzer: SentimentAnalyzer  
) : ViewModel() {  
    val articles = Pager(  
        config = PagingConfig(pageSize = 20),  
        pagingSourceFactory = { NewsPagingSource(service, "YOUR_API_KEY", analyzer) }  
    ).flow.cachedIn(viewModelScope)  
}
```

Adapter with sentiment analysis

```
class NewsAdapter : PagingDataAdapter<ArticleWithSentiment, NewsViewHolder>(ARTICLE_COMPARATOR)  
{  
  
    override fun onBindViewHolder(holder: NewsViewHolder, position: Int) {  
        val item = getItem(position)  
        item?.let {  
            holder.bind(it.article)  
  
            // Update UI based on sentiment  
            val sentiment = it.sentiment  
            val sentimentColor = if (sentiment.positive > sentiment.negative) {  
                Color.GREEN  
            } else {  
                Color.RED  
            }  
  
            holder.itemView.setBackgroundColor(sentimentColor.copy(alpha = 0.2f))  
        }  
    }  
}
```

Exercise 1 – Advance with sentiment analysis



Sentiment filter

```
val positiveArticles = articles.map { pagingData ->
    pagingData.filter { it.sentiment.positive > 0.7 }
}
```

Sentiment History

```
// Track sentiment over pages
val sentimentHistory = mutableStateListOf<Float>()
articles.onEach { pagingData ->
    pagingData.map { it.sentiment.positive }.collect { sentiment ->
        sentimentHistory.add(sentiment)
    }
}
```

Homework— Image Gallery with Object Detection



Build.gradle

```
// Paging
implementation "androidx.paging:paging-runtime-ktx:3.2.1"

// Retrofit
implementation "com.squareup.retrofit2:retrofit:2.9.0"
implementation "com.squareup.retrofit2:converter-gson:2.9.0"

// ML Kit (Image Labeling)
implementation "com.google.mlkit:image-labeling:17.0.7"

// Coil for images
implementation "io.coil-kt:coil:2.5.0"

// Lifecycle components
implementation "androidx.lifecycle:lifecycle-runtime-ktx:2.6.2"
```


Homework– Image Gallery with Object Detection



1. Access to this link, register and get api: <https://pixabay.com/api/docs/>
2. Create data classes (Hint: PixabayResponse, ImageItem, ProcessedImage)
3. Image Processor:

```
class ImageAnalyzer(context: Context) {  
    private val labeler = ImageLabeling.getClient(ImageLabelerOptions.DEFAULT_OPTIONS)  
  
    suspend fun analyzeImage(imageUrl: String): List<String> = coroutineScope {  
        try {  
            val bitmap = Coil.execute(ImageRequest.Builder(context)  
                .data(imageUrl)  
                .allowHardware(false)  
                .build()).drawable?.toBitmap()  
  
            val image = InputImage.fromBitmap(bitmap!!, 0)  
            val labels = labeler.process(image).await()  
  
            labels.take(3).map { it.text }  
        } catch (e: Exception) {  
            emptyList()  
        }  
    }  
}
```

Homework— Image Gallery with Object Detection



Paging source:

```
class ImageAnalyzer(context: Context) {  
    private val labeler = ImageLabeling.getClient(ImageLabelerOptions.DEFAULT_OPTIONS)  
  
    suspend fun analyzeImage(imageUrl: String): List<String> = coroutineScope {  
        try {  
            val bitmap = Coil.execute(ImageRequest.Builder(context)  
                .data(imageUrl)  
                .allowHardware(false)  
                .build()).drawable?.toBitmap()  
  
            val image = InputImage.fromBitmap(bitmap!!, 0)  
            val labels = labeler.process(image).await()  
  
            labels.take(3).map { it.text }  
        } catch (e: Exception) {  
            emptyList()  
        }  
    }  
}
```

Homework– Image Gallery with Object Detection



View model, adapter: Code by yourself

Smart search

```
// Filter images by AI tags
val searchQuery = MutableStateFlow("")
val filteredImages = images.combine(searchQuery) { pagingData, query ->
    pagingData.filter { processedImage ->
        processedImage.aiTags.any { it.contains(query, true) }
    }
}
```

```
// Room database for storing AI results
@Dao
interface ImageAnalysisDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAnalysis(result: AnalysisResult)

    @Query("SELECT tags FROM analyses WHERE imageId = :id")
    suspend fun getAnalysis(id: Int): List<String>?
}
```

AI Cache



References

- <https://web.stanford.edu/>
- <https://developer.android.com/>
- <https://introcs.cs.princeton.edu/>
- <https://www.cs.utexas.edu/>
- <https://www.codepath.com/>



Thank you for listening

*"Coming together is a beginning;
Keeping together is progress;
Working together is success."*

- HENRY FORD